Interrupções e Barramento

Kinan Principe

29 abr. 2026

Contents

1 Interrupções **3**
1.1 Programa 3
1.2 Timer 3
1.3 Entrada/Saída 3
1.4 Hardware 3
1.5 Execução sem Interrupções (polling) 4
1.6 Execução com Interrupções 5

2 Ciclo de Instrução com Interrupção **5**
2.1 Tratador de Interrupção 6
2.2 Overhead 6

3 Interrupções Simultâneas **6**
3.1 Por Prioridade 6

4 Estrutura de Interconexão **7**
4.1 Barramento 7
4.2 Operação do Barramento 7
4.3 Limitações do Barramento 8

5 Principais tipos de Arbitragem **8**
5.1 Arbitragem Centralizada 8
5.2 Arbitragem Distribuída 9

6 Interconexão Ponto a Ponto (Point-to-Point) **9**

1 Interrupções

Mecanismo que interrumpe o fluxo de execução da CPU, de modo que eventos externos sejam tratados imediatamente.
Essencial para sistemas multitarefa.
Existem vários tipos de interrupções, a depender da origem ou resultado pretendido.

1.1 Programa

Ocorre durante execução de instruções e são causadas por erros ou condições especiais. Exemplos:

- Divisão por zero;
- Instrução inválida;
- Acesso ilegal à memória.

1.2 Timer

Geradas por um *temporizador interno* e permitem o controle do sistema operacional. Exemplos:

- Troca de tarefas;
- Atualização do sistema a cada intervalo de tempo.

1.3 Entrada/Saída

Geradas por dispositivos externos e indicam o *término* ou *erro* de operação. Exemplos:

- Impressora terminou a impressão;
- Teclado recebeu uma tecla;
- Disco concluiu leitura.

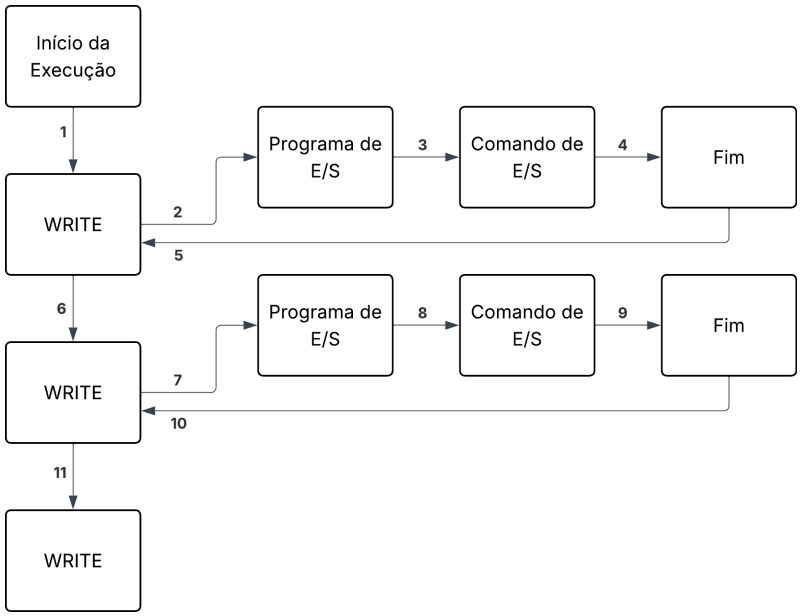
1.4 Hardware

Causadas por *falhas físicas* ou *problemas no sistema*. Exemplos:

- Falta de energia;
- Erro de memória;
- Superaquecimento.

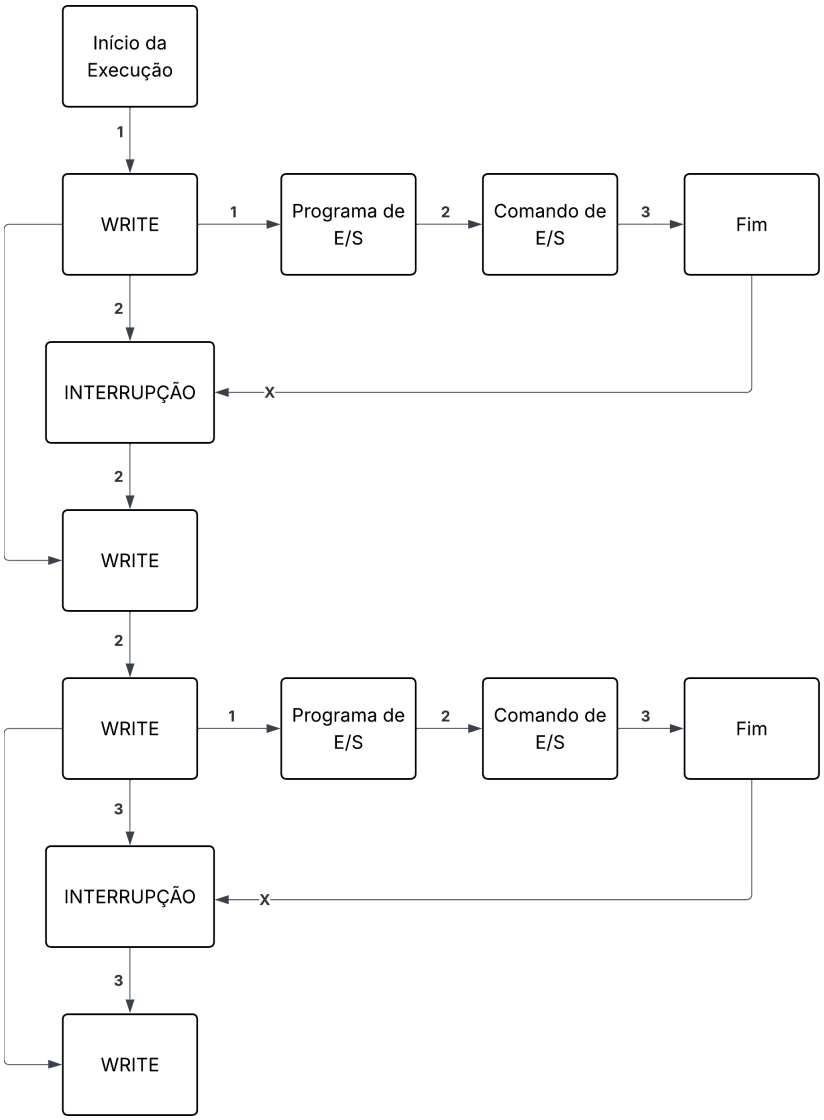
1.5 Execução sem Interrupções (polling)

Sem interrupções, um sistema deve aguardar uma operação terminar antes de começar outra, causando drástico impacto em sua eficiência.



1.6 Execução com Interrupções

Com interrupções, um sistema inicia uma operação de entrada e saída e continua executando as outras instruções até receber uma interrupção (dispositivo sinalizando que terminou).

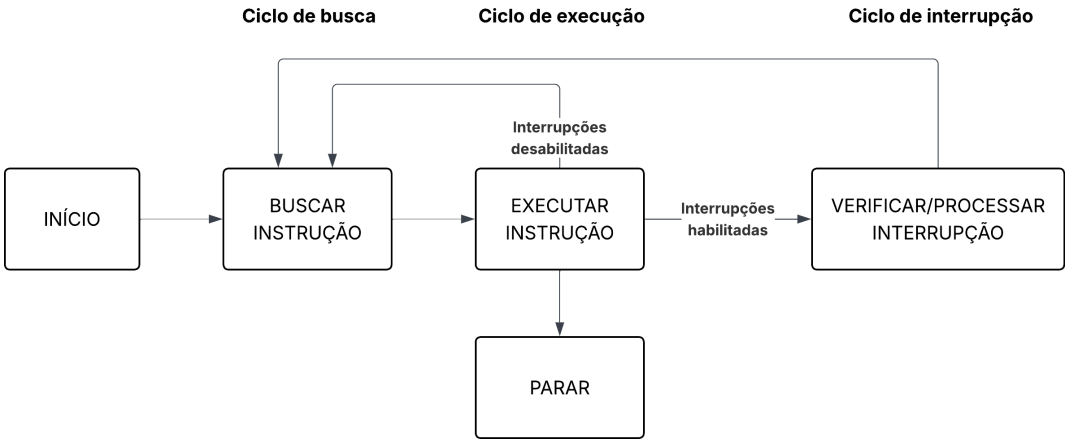


2 Ciclo de Instrução com Interrupção

O ciclo de execução das instruções com interrupções habilitadas seguem a sequência:

1. Busca (fetch);

2. Execução (execute);
3. Verificação de interrupção.



2.1 Tratador de Interrupção

Também chamado de *ISR* (Interrupt Service Routine), é um código do sistema operacional que identifica a origem da interrupção e executa a ação necessária.

2.2 Overhead

O *overhead de interrupções* é o tempo e recursos computacionais gastos para *salvar contexto*, *executar ISR* e *restaurar o contexto*. Em suma, é o custo do uso das interrupções.

3 Interrupções Simultâneas

Vários dispositivos poderam gerar múltiplas interrupções ao mesmo tempo e existem diversas maneiras de como o sistema decide qual é que será efetuada.

3.1 Por Prioridade

Uma das maneiras é definindo para cada interrupção um nível de prioridade, podendo até determinada interrupção mais importante interromper uma outra interrupção. Exemplo:

- Comunicação (alta prioridade);
- Disco (média prioridade);
- Impressora (baixa prioridade).

4 Estrutura de Interconexão

Conecta os seguintes componentes do sistema: *CPU*, *Memória* e *Dispositivos de Entrada e Saída*.

4.1 Barramento

Conjunto de linhas paralelas, onde cada linha transporta um *bit*.
O barramento é um meio de comunicação compartilhado que comunica múltiplos dispositivos e permite a troca de dados entre eles. Apenas um dispositivo transimite por vez, pois senão pode gerar conflitos.

- **Barramento de Dados:** Responsável por transportar dados entre *CPU*, *Memória* e *Dispositivos de Entrada e Saída*.
  - A quantidade de bits transmitidos depende da largura do barramento (8, 16, 32, 64 bits).
- **Barramento de Endereço:** Indica onde o dado deve ir/vir, ou seja, uma posição na memória ou dispositivo. Sua largura determina a capacidade máxima de memória.
  - Por exemplo, um barramento de *32 bits* pode endereçar até *4GB* de memória.
- **Barramento de Controle:** Responsável por controlar e sincronizar as operações com sinais como: *READ*, *WRITE*, *CLOCK*, *INTERRUPT*.
  - A *CPU* pode, por exemplo, enviar um sinal *READ* para ler um determinado dado da memória.

4.2 Operação do Barramento

Os sistemas computacionais fazem uso da *arbitragem* para organizar o acesso ao barramento pelos dispositivos e evitar conflitos (dois ou mais dispositivos acessando o barramento simultanemanete).

A *arbitragem* ocorre como um conjunto de etapas na seguinte ordem:

1. **Solicitação:** O Dispositivo pede o controle do barramento para o sistema (*bus reques*).
2. **Concessão:** O Sistema decide quem pode usar o barramento (*bus grant*).
3. **Transferência de Dados:** O Dispositivo envia:
   - *Endereço* (Onde acessar);
   - *Dados* (O que transferir);
   - *Controle* (leitura/escrita);

4. **Confirmação:** O Dispositivo destino confirma a operação com o sinal **ACK** (acknowledge).

5. **Liberação do Barramento:** Após o uso, dispositivo libera o barramento para outro dispositivo utilizar.

4.3 Limitações do Barramento

- **Gargalo de Desempenho**: Apenas um dispositivo por vez pode acessá-lo e os outros devem esperar;
- **Concorrência e Conflitos**: Múltiplos dispositivos disputam o uso, podendo ocorrer *colisão de sinais*;
- **Escalabilidade Limitada**: Quanto mais dispositivos conectados, pior o desempenho e maior o tempo de espera.
- **Limitação Física e Elétrica**: Dificuldade de aumentar a velocidade e problemas de sincronização em altas frequências;
- **Latência maior**: O processo de solicitar acesso, aguardar liberação e realizar a operação é demorado;

5 Principais tipos de Arbitragem

5.1 Arbitragem Centralizada

Nesse modelo existe um árbitro central, que geralmente é a *CPU* ou um *controlador*.

- **Prioridade Fixa:** Cada dispisitivo tem uma prioridade definida e o de maior prioridade sempre ganha.
  - Modelo simples, no entanto, pode acarretar em *starvation* em processos mais complexos, onde dispositivos de baixa prioridade podem nunca conseguirem acesso ao barramento.
- **Prioridade Rotativa:** O acesso alterna entre os dispositivos.
  - Justo mas não é eficiente atendendo urgências.
- **Prioridade Dinâmica:** A prioridade muda com o tempo ou situação, por exemplo, dispositivos com muito tempo de espera ganham prioridade ou dispositivos críticos recebem propridade temporária.
  - Modelo mais eficiente;
  - Evita starvation.

5.2 Arbitragem Distribuída

Não existe um árbitro central e os dispositivos decidem entre si.

- **Auto-seleção:** Todos colocam seu identificador no barramento e o maior ou menor valor vence.
  - Rápido e paralelo.
- **Detecção de Colisão:** Dispositivos tentam usar o barramento e, se houver conflito, tentam novamente.
  - Simples mas pode gerar atrasos.
- **Token Passing:** Um token (permissão) circula entre os dispositivos e só quem o tem pode acessar o barramento.
  - Organizado e sem conflitos mas pode ser mais lento.

6 Interconexão Ponto a Ponto (Point-to-Point)

Componentes são ligados diretamente entre si e cada conexão é dedicada.

- Alta velocidade, menor latência;
- Maior escalabilidade;
- Comunicação feita por *pacotes de dados*.

Exemplos: PCI Express (placas de vídeo, SSDs), Quick Path Interconnect - QPI (comunicação entre múltiplos processadores).